GEPA: Reflective Prompt Optimization

Large Language models are highly sensitive to the instructions provided in their prompts. A weak or underspecified prompt can cause a capable model to produce incorrect formats, misunderstand business rules or behave inconsistently across different examples. Traditional prompt engineering addresses this problem manually: a developer examines failures, modifies the prompt, test the new version and repeats the process.

GEPA is a reflective prompt optimizer that can optimize either an individual LLM prompt or the prompts belonging to multiple LLM modules in a compound AI system. Instead of modifying model weights, GEPA keeps the underlying language models fixed and searches for better natural-language instructions.

A compound system can be presented as

$$\Phi = (M, C, X, Y)$$

Where $M = (M_1, ….., M_n)$ represents the language model modules, C represents the control flow between them, and X and Y represent the global input and output schemas. Each modules M_i has an associated prompt π_i and GEPA optimizes the collection of prompts rather than retraining the underlying models.

GEPA working principle

GEPA requires two main inputs:

1. A dataset containing task inputs and the expected outputs or evaluation metadata.
2. An initial candidate, or seed prompt P_0, that GEPA will attempt to improve.

I used the seed prompt-

```
You are a helpful assistant. Answer the user's question.
```

Will got through an example to under how GEPA build its score matrix, identifies the Pareto Frontier and continues the optimization loop.

For this example, I made a use case for a customer-support triage problem where an LLM needs to convert a free-text customer message into an exact JSON structure without me giving extra specific instructions:

```
{
    “category”: “damaged”,
    “order_id”: “ORD-4088”,
    “priority”: “normal”
}
```

To further clarify, one of the instances of the dataset looked like this-

```
{
    "input": "I thought ORD-1004 was damaged, but it works perfectly. I was actually billed twice",
    "answer": {
        "category": "billing",
        "order_id": "ORD-1004",
        "priority": "normal"
    }
}
```

So, the input is a free form text from the customer and the LLM is expected to return the answer.

The dataset is divided into 2 parts-

- $D_{feedback}$ (Used for training. For minibatch rollouts, diagnostic feedback, reflection, and prompt mutation)
- D_{pareto} (Used for validation. To evaluate accepted candidates and maintain instance-level performance information for candidate selection)

Step 1: Creating the Initial Scores Matrix

A **task instance** is one example from the dataset. In this project, one instance consists of a customer message x_i and the corresponding expected JSON output y_i.

Suppose there is a validation set D_{pareto} with 4 task instances (I_1, I_2, I_3, I_4) and the task is to optimize the system.

At the start, GEPA evaluates the seed prompt candidate (P_0) across all 4 instances. The result forms Row 0 of the Scores Matrix (S).

Candidate prompt	Instance I_1	Instance I_2	Instance I_3	Instance I_4	Mean Score
P_0(Seed)	1.0	0.0	0.0	0.0	0.25

The values in the score matrix come from the evaluation function. For each validation instance, GEPA runs the task LLM using the candidate prompt P_k and the input x_i. The task LLM produces a predicted output $\hat{y}_i$. The evaluator then compares this predicted output against the expected answer y_i and assigns a score. The expected answer is used by the evaluator and it is not provided to the task LLM.

As GEPA runs a few mutations, it evaluates new prompt candidates (P_1, P_2, P_3) on D_{pareto} and appends them to the matrix.

Candidate prompt	Instance I_1	Instance I_2	Instance I_3	Instance I_4	Mean Score
P_0 (Seed)	1.0	0.0	0.0	0.0	0.25
P_1	0.0	1.0	0.0	0.0	0.25
P_2	1.0	1.0	0.0	0.0	0.50
P_3	0.0	0.0	1.0	0.0	0.25

The next step illustrates how GEPA can select an existing candidate, such as P3 and mutate it into a new candidate P4.

Note: The above P_0–P_4 example is a simplified hypothetical example used to explain GEPA's candidate-selection mechanism. Later sections will show the actual candidates and results generated by the notebook implementation.

Step 2: Pareto Filtering and Selection

Standard optimizers would simply pick P_2 because it has the highest overall average score (0.50). GEPA avoids this greedy trap by performing Pareto-based filtering across individual instances.

1. Identify Best Candidate(s) per task Instance
   GEPA checks the maximum score achieved for each column $s^{[i]}$:
   - I_1: Max score = 1.0 → achieved by $\{P_0, P_2\}$
   - I_2: Max score = 1.0 → achieved by $\{P_1, P_2\}$
   - I_3: Max score = 1.0 → achieved by $\{P_3\}$
   - I_4: Max score = 0.0 → all candidates tie at 0.0, so $\{P_0, P_1, P_2, P_3\}$
2. Prune strictly dominated Candidates
   - P_2 VS P_0: P_2 scores 1.0 on I_1, I_2 while P_0 scores 1.0 on I_1 and 0.0 on I_2. P_2 strictly dominates P_0. So P_0 is pruned.
   - P_2 VS P_1: P_2 scores 1.0 on I_1, I_2 while P_1 scores 1.0 on I_2 and 0.0 on I_1. P_2 strictly dominates P_1, so P_1 is pruned.
   - P_2 VS P_3: P_2 fails I_3(0.0), but P_3 solves I_3 (1.0). Neither candidate dominates the other

   Key Insight: Even though P_3 has a lower global average score (0.25), it is preserved in the pool because it contains a unique "winning" strategy for task instance I_3.

3. Frequency Weighted Stochastic Sampling
   GEPA counts how many instances each candidate leads in ($f[P_k]$)
   - $f[P_2] = 3$ (leads on I_1, I_2, I_4)
   - $f[P_3] = 2$ (leads on I_3, I_4)

   GEPA samples a candidate proportional to its frequency (60% probability for P_2, 40% for P_3). Suppose it samples P_3

Step 3: How GEPA algorithm continues

Once P_3 is selected for mutation, the loop executes the following steps-

1. GEPA runs P_3 on a small minibatch from $D_{feedback}$ (3training questions). Calculates a evaluation score σ and provides feedback on whether it matched with actual answers or why it did not match.
2. There is a function that creates a reflective dataset comprising of input, answer, full_assistant_response, feedback.
3. A reflection LM inspects P_3 the execution traces (input, answer, full_assistant_response), textual feedback. It rewrites P_3 into an updated candidate prompt P_4.
4. GEPA runs P_4 on the exact same minibatch and calculates score σ`.
5. If P_4 average minibatch score σ` is greater than σ, GEPA proceeds to full validation otherwise P_4 is discarded immediately.
6. Assuming P_4 succeeds, GEPA runs it across all validation instances (I_1, I_2, I_3, I_4). Suppose P_4 achieves scores [1.0, 0.0, 1.0, 1.0] (Mean = 0.75)
   GEPA appends P_4 to the Scores Matrix and records P_3 as its parent in the Ancestry Tree (A)

Candidate Prompt	Instance I_1	Instance I_2	Instance I_3	Instance I_4	Mean Score	Parent Node
P_0 (Seed)	1.0	0.0	0.0	0.0	0.25	None
P_1	0.0	1.0	0.0	0.0	0.25	P_0
P_2	1.0	1.0	0.0	0.0	0.50	P_0
P_3	0.0	0.0	1.0	0.0	0.25	P_0
P_4	1.0	0.0	1.0	1.0	0.75	P_3

In the next cycle, P_4 now leads on 3 instances (I_1, I_3, I_4) shifting the search towards higher performance while retaining P_2 for its unique strength on I_2. The loop continues until the rollout budget is exhausted, returning the single highest average scoring prompt.

Actual GEPA Implementation

The previous P_0, P_1, P_2, P_3 and P_4 example was used to understand the overall GEPA algorithm. I then implemented GEPA on a simple customer-support classification problem to see how the reflective optimization process works in practice.

The objective was to take a free-form customer-support message and return a JSON object containing exactly three fields:

```
{
    "category": "damaged",
    "order_id": "ORD-4088",
    "priority": "normal"
}
```

The possible values for category were:

- damaged
- billing
- delivery
- refund

The order_id was either an order identifier in the form ORD-#### or null if no order ID was provided.

The priority was either high or normal.

For example:
Input: Urgent, I was charged twice for ORD-2065
Expected output:

```
{
    "category": "billing",
    "order_id": "ORD-2065",
    "priority": "high"
}
```

The initial dataset contained:

- 6 feedback examples
- 4 validation examples
- 2 final test examples

The feedback examples were used by GEPA for minibatch execution, reflection and prompt mutation. The validation examples were used to evaluate accepted candidates and maintain candidate performance. The final test examples were kept separate and were used only after optimization to compare the original and optimized prompts.

I intentionally started with a very generic system prompt:

```
You are a helpful assistant. Answer the user's question.
```

This was intentional. The goal was to see whether GEPA could learn the required behavior through evaluator feedback and reflection rather than manually placing all of these rules inside the initial prompt

To optimize the prompt, GEPA needs an evaluator that can compare the output produced by the LLM with the expected answer.
For each example:

$$(x_i, P_k) \rightarrow \text{Task LLM} \rightarrow \hat{y}_i$$

Where:

- x_i = customer message
- P_k = candidate prompt
- $\hat{y}_i$ = generated response

The evaluator then compares $\hat{y}_i$ with expected answer y_i and returns score and evaluation feedback.

This score tells GEPA how well the candidate performed.

The feedback provides textual information that can later be given to Reflection LM.

The evaluator that I had built followed the logic below-

Case 1: Output is not valid JSON

If the model response could not be parsed as JSON then score = 0 and the feedback looked like:

```
Invalid JSON response: Expecting value...
```

Case 2: JSON contains the wrong keys

If the response was valid JSON but did not contain exactly the same keys as the expected output: then score = 0 and the feedback identified the expected and received keys.

Case 3 Correct JSON schema

If the response contained the correct three keys, the evaluator compared the individual fields:

The score was: score = $\frac{Number\ of\ correctly\ predicted\ fields}{Total\ number\ of\ fields}$

Since there were three fields:

Score = $\frac{correct\ category+corrwect\ order\ ID+correct\ priority}{3}$

For example,

if category and order_id were correct but priority was incorrect:

score = $\frac{2}{3}$ = 0.67

A complete correct result received score = 1

Evaluating the Seed Prompt

I first ran P_0 on a minibatch containing three feedback examples:

For the input:

“My order ORD-1042 arrived broken.”

The expected output was:

```
{
    “category”: “damaged”,
    “order_id”: “ORD-1042”,
    “priority”: “normal”
}
```

However, using the generic seed prompt, the LLM responded approximately as a normal customer-support assistant:

```
I'm sorry to hear that your order ORD-1042 arrived broken. Please provide me with some additional details or photos of the damaged item, and I will assist you with a replacement or refund.
```

The response is reasonable conversationally, but it does not satisfy the application requirement because it is not JSON.

Therefore, score = 0

The same behavior occurred for the other examples in the minibatch.

Letting GEPA reflect on the Failures

The next step was to allow GEPA's Reflection LM to inspect the failures and generate a new prompt automatically.

Conceptually, the Reflection LM receives information similar to:

```
Current prompt:
You are a helpful assistant. Answer the user's question.

Input:
My order ORD-1042 arrived broken.

Expected output:
{"category": "damaged",
 "order_id": "ORD-1042",
 "priority": "normal"}

Generated output:
I'm sorry to hear that your order arrived broken...

Evaluator feedback:
Invalid JSON response...
```

The Reflection LM then asks, in effect:

- Why did the current prompt fail, and what instructions should be added to prevent the same failure?"

It uses this information to propose a new candidate prompt. This is the reflective mutation step:

$$P_0 \xrightarrow{Reflection} P_1`$$

Where $P_1`$ is the proposed mutation

However, an important problem appeared during this first experiment.

Problem with the Initial Evaluator Feedback

The initial evaluator was sufficient for **scoring the response**, but its feedback was not sufficiently informative for **optimizing the prompt**.

For example, when the model returned conversational text instead of JSON, the Reflection LM mainly received feedback such as:

```
Invalid JSON response:
Expecting value: line 1 column 1
```

This tells the Reflection LM that the response was not valid JSON.

But it does **not clearly tell it what JSON structure is actually required**.

The feedback did not explicitly describe:

- the exact required keys
- the allowed category values
- the order-ID format
- the rule for determining priority
- the category-precedence rules

As a result, the Reflection LM correctly understood that the application needed structured customer-support JSON, but it had to infer the desired schema itself.

It proposed a much more elaborate customer-support prompt containing fields such as:

- intent
- issues
- orderId
- urgency
- priority
- requestedOutcome
- routingQueue
- requiredCustomerInfo
- agentActions
- customerFacingMessage
- confidence

This was a reasonable customer-support schema. But it was **not the schema I had**.

The evaluator expected exactly:

```
{"category": "…..",
 "order_id": "…..",
 "priority": "……"}
```


Therefore, the proposal mutation still received score = 0

Since its minibatch score was not better than the parent candidate:

$\sigma(P_1`) \not\succ \sigma(P_0)$

GEPA rejected the mutation. The candidate was never added to the candidate pool.

Therefore, the best candidate remained P_0.

This was an important observation from the experiment.

The optimizer was functioning correctly. The problem was that the feedback supplied to the Reflection LM did not contain enough information about the actual task requirements.

Improving Evaluator Function

The scoring logic was largely retained, but the evaluator feedback was expanded to explicitly describe the required behavior.

I created a required_schema description containing the important task rules:

```
You must produce ONLY a valid JSON object containing
exactly these three keys:

category
order_id
priority

Allowed category values:
damaged, billing, delivery, refund.

If damage and delivery issues coexist, choose damaged.
Extract order_id as ORD-####, or null if absent.
Set priority to high if the message explicitly says
URGENT; otherwise normal.

Do not include additional keys.
```

Now, if the output contained the wrong keys, the evaluator did not simply say:

```
Incorrect JSON keys.
```


Instead, it could return feedback similar to:

```
Incorrect output schema.

Expected keys:
['category', 'order_id', 'priority']

Received keys:
[...]

You must produce ONLY a valid JSON object containing
exactly these three keys...

Allowed category values:
damaged, billing, delivery, refund...

Extract order_id as ORD-####...

Set priority to high if the message explicitly says
URGENT...
```

Likewise, if the response was not valid JSON, the feedback included the required schema and an example of the correct output.

This did **not give the correct prediction for every new customer message directly**. Instead, it described the general rules that define the task.

The Reflection LM could now infer much more precisely what was missing from P_0.

The new candidate became P_1.

The minibatch performance improved, so the candidate passed the first verification step.

GEPA then evaluated P_1 across the complete validation set.

The results were:

Candidate	I_1	I_2	I_3	I_4	Mean
P_0	0.0	0.0	0.0	0.0	0.0
P_1	1.0	1.0	1.0	1.0	1.0

The ancestry tree was simply: $P_0 \rightarrow P_1$

Because P_1 performed at least as well as P_0 on every validation instance and strictly better on all four, P_1 dominated P_0.

After this point, GEPA repeatedly selected P_1, but the sampled examples already had perfect scores. There was therefore no failure information from which to generate another useful mutation.

As a result, the optimization effectively stopped after the first successful candidate.

Refer to Appendix A for the optimized prompt from GEPA.

Final Test Set

The final step was to compare P_0 and P_1 on examples that had not been used for feedback or validation.

The two test examples were:

```
URGENT: ORD-8080 arrived damaged.
{
   "category": "damaged",
   "order_id": "ORD-8080",
   "priority": "high"
}
```

```
I would like my money back for ORD-9090.
{
   "category": "refund",
   "order_id": "ORD-9090",
   "priority": "normal"
}
```

The original seed prompt produced scores: [0.0,0.0]. Mean test Score (P_0) = 0.0

The optimized prompt produced: [1.0,1.0]. Mean test Score (P_1) = 1.0

Therefore, the optimized prompt also achieved perfect performance on both held-out test examples.

References

1. L. A. Agrawal *et al.*, "GEPA: Reflective prompt evolution can outperform reinforcement learning," *arXiv preprint arXiv:2507.19457*, 2025.

Appendix A

```
You are a triage assistant that converts a single free-form customer message into a strict JSON record.

Output requirements:
- Produce ONLY a single valid JSON object with exactly these three keys: "category", "order_id", "priority".
- Do not include any other text, explanations, or keys.
- Use double quotes for keys and string values. Use the JSON null literal (unquoted) when applicable.
- Recommended key order: category, order_id, priority.

Field rules:
1) category (string; one of: damaged, billing, delivery, refund)
  - damaged: Physical issues with the item (e.g., damaged, broken, cracked, dented, defective, smashed, leaking). If both damage and delivery issues are mentioned, choose damaged.
  - delivery: Shipping/arrival problems (e.g., late, delayed, not delivered, missing package, lost, delivery issue).
  - billing: Payment/charge problems (e.g., charged twice, overcharged, billing error, incorrect charge).
  - refund: An explicit request for a refund (e.g., refund, money back).
  Choose the single best-fitting category from the above list.

2) order_id (string or null)
  - Extract the first occurrence matching the pattern ORD-#### (exactly "ORD-" followed by four digits), e.g., via regex \bORD-\d{4}\b.
  - If no such ID is present, set order_id to null.

3) priority (string; one of: high, normal)
  - Set to high only if the message explicitly contains the word "URGENT" (case-insensitive).
  - Otherwise, set to normal.

Examples of correct outputs:
- {"category": "damaged", "order_id": "ORD-4088", "priority": "normal"}
- {"category": "billing", "order_id": "ORD-2055", "priority": "high"}
- {"category": "refund", "order_id": null, "priority": "high"}
```